Ruby includes support for rational and complex numbers. Rational numbers are the ratio of two integers—they are fractions—and hence have an exact representation (unlike floats). Complex numbers represent points on the complex plane. They have two components, the real and imaginary parts.

Ruby has a literal syntax for both Rational and Complex numbers, but it might not be what you expect. Rationals first. If you try to represent a fraction like, say `3/4` directly, Ruby will interpret that as integer division and return `0`. To make the fraction into a Ruby Rational instance, you need to add the letter `r`, as in `3/4r`. You can convert decimals into Rationals with this syntax, for example `0.75r` will also convert into three-fourths (but note that just `.75r` is still a syntax error, because decimal numbers need to have digits on both sides of the decimal point). Strings can be converted to rationals with the `to_r` method. Finally, Ruby offers a conversion method, `Rational`, which takes either a string or two arguments and creates a Rational instance.

```
3/4             # => 0
3/4r            # => (3/4)
0.75r           # => (3/4)
"3/4".to_r      # => (3/4)
Rational(3, 4)  # => (3/4)
Rational("3/4") # => (3/4)
```

The literal syntax for Complex numbers uses `i` as a suffix, alternately you can convert strings via the `to_c` method, and there is a `Complex` conversion method.

```
1+2i            # => (1+2i)
"1+2i".to_c     # => (1+2i)
Complex(1, 2)   # => (1+2i)
Complex("1+2i") # => (1+2i)
```

If, in fact, you were wondering, a number can be both rational and imaginary, `5.7ri`, but the `r` needs to come before the `i` otherwise you get a syntax error.

All numbers are objects and respond to a variety of messages. So, unlike, say, Python, you find the absolute value of a number by writing `num.abs`, not `abs(num)`.

Finally, we'll offer a warning for users of other languages. Strings that contain just digits are *not* automatically converted into numbers when used in expressions. This tends to bite when reading numbers from a file or when trying to use the parameters from a web request.

For example, we may want to find the sum of the two numbers on each line for a file such as the following:

```
3 4
5 6
7 8
```

The following code doesn't work:

```
some_file.each do |line|
  v1, v2 = line.split    # split line on spaces
  print v1 + v2, " "
end
```

produces:

```
34 56 78
```